

Universidad del Quindío

Facultad de Ingeniería

Programación III

2026-2

Parcial 1 programación III

Presentado por:

Juan Manuel Castaño Buitrago

Christian David Lopera Castaño

Samuel Mejía Guerrero

Profesor:

Carlos Andrés Flórez Villarraga

Armenia, Quindío

Octubre 2026

PARTE A. DISEÑO:

1. Los recolectores, lotes y pesajes se mantienen inicialmente como listas. Esto facilita el uso de funciones del módulo `enum` para iterar y procesar datos de forma funcional. Facilitan el filtrado, mapeo y acumulación global de datos usando las funciones puras del módulo `enum`. Se pierde la eficiencia en la búsqueda directa. Para calcular las bonificaciones por productividad (Regla 3) y los descuentos de alimentación (Regla 4), los pesajes de cada recolector se agrupan por día en un mapa, al agrupar por día es muy fácil sumar los kilos recolectados en un día específico usando `Enum.reduce`. Además, para saber cuántos días trabajó la persona, simplemente se consulta el tamaño del mapa con `map_size`. Se pierde el orden secuencial original de los pesajes, pero no afecta en nada porque el cálculo requiere agrupaciones acumulativas y no secuenciales.

2. Funciones impuras:

`Util.leer`: Imprime un mensaje y detiene el programa esperando entrada del teclado.

`Util.imprimir_mensaje`: Escribe texto en la consola.

`Programa.main`: Orquesta toda la ejecución y llama a las impresiones.

`Programa.procesar_linea`: Utiliza impresiones en consola para avisar si el pesaje fue agregado o rechazado.

`Programa.imprimir_desprendible`: Busca un recolector e imprime en consola su pago.

Funciones puras:

En Datos: recolectores, lotes, pesajes.

En validación: `validar_pesaje` y todas sus funciones auxiliares (`validar_recolector`, `validar_lote`, `validar_dia`, `validar_kilos`, `validar_verdes`).

En liquidación: `valor_pesaje`, `bonificacion_dia`, `descuento_alimentacion`, `liquidar_recolector` y `liquidar_todos`.

En reportes: Todas las funciones que generan los reportes (`r1_rechazos` hasta `r8_todos_los_lotes`), además de `ranking`, `combinar_fincas` y `mapa_kilos_por_dia`.)

3. Validación: Contiene la lógica pura que verifica si un pesaje cumple con las 5 reglas de negocio.

Datos: Su única función es retornar las listas de datos originales.

Liquidación: Contiene las fórmulas puras de negocio (cálculo de valor por pesaje, bonificaciones diarias, descuentos y total neto).

Reportes: Genera el texto o cadenas de caracteres (String) formateadas de cada reporte sin imprimirlos.

Util: Reduce el código llamando funciones que se muestran en la consola.

Programa: Coordina la ejecución, llama a la validación, procesa la entrada del usuario y delega la impresión.

Parte B. Construcción del programa

El programa se ha estructurado y construido cumpliendo estrictamente con los requerimientos técnicos exigidos para el parcial:

B.1 Organización: Se implementaron de forma independiente los módulos datos, validación, liquidación, reportes, útil y el archivo ejecutable programa.exe con su respectiva función main. Las funciones puras de reportes retornan cadenas de texto sin realizar impresiones directas en consola.

B.2 Validación: Las reglas de negocio de rechazo se encadenaron utilizando la estructura with para mantener la legibilidad y evitar condicionales anidados.

B.3 Liquidación: Los cálculos del valor del pesaje, las bonificaciones diarias y los descuentos de alimentación se aislaron en funciones puras independientes.

B.4 Reportes y explicación del reporte 6 (Mejor calidad): Se generaron exitosamente los ocho reportes estadísticos requeridos apoyándose en funciones del módulo enum y map.

Explicación del Reporte 6: Porcentaje Ponderado vs Promedio Simple

¿Por qué el porcentaje ponderado es distinto al promedio simple?

R/ El promedio simple suma los porcentajes de verdes de cada pesaje y los divide entre el número total de transacciones, tratando un pesaje pequeño de la misma forma que uno grande. Por el contrario, el porcentaje ponderado multiplica el porcentaje de verdes de cada bulto por sus respectivos kilos, dándole mayor peso matemático a las recolecciones más grandes y reflejando la calidad real de toda la masa de café recolectada.

Caso evidente en nuestros datos de prueba: El recolector Jhon Fredy Castaño (R02) cuenta con 5 pesajes válidos con diferentes volúmenes y calidades:

100 kg con un 2% de verdes.

110 kg con un 1% de verdes.

Pesajes menores de 45 kg, 60.5 kg y 65 kg con 3%, 4% y 5% de verdes respectivamente.

Si se calculase un promedio simple, el resultado sería la suma aritmética de los porcentajes dividida entre cinco: $(2 + 1 + 3 + 4 + 5) / 5 = 3.0\%$. Sin embargo, al aplicar la fórmula del porcentaje ponderado por kilos ($\text{suma}(\text{verdes} \times \text{kilos}) / \text{suma}(\text{kilos})$), el resultado obtenido por el programa es de 2.66%. La diferencia evidencia que los pesajes más pesados de Jhon Fredy tuvieron una excelente calidad (1% y 2%), lo que arrastra favorablemente el promedio real hacia abajo, demostrando que su desempeño global fue superior al que arrojaría un simple cálculo aritmético.

B.5 Interacción con el usuario: El sistema procesa correctamente la entrada de un pesaje adicional mediante lectura por consola, evaluando su formato con tuplas y aplicando las validaciones correspondientes antes de integrarlo en los cálculos y reportes globales, finalizando con la consulta dinámica del desprendible de pago por código de recolector.

Parte C. Investigación

C.1. Opciones con keyword lists:

A continuación, se presentan las salidas generadas por el programa para cada una de las tres pruebas solicitadas desde la función principal:

```
Prueba 1: Reportes.ranking(liquidaciones, [])
[
  %{
    recolector: %{
      codigo: "R02",
      nombre: "Jhon Fredy Castaño",
      alimentacion: false
    },
    detalle_diario: [
      %{kilos: 145, dia: 1, valor: 150000.0, bonificacion: 8000},
      %{kilos: 125.5, dia: 2, valor: 125500.0, bonificacion: 8000},
      %{kilos: 110, dia: 3, valor: 115500.0, bonificacion: 0}
    ],
    suma_pesajes: 391000.0,
    total_bonos: 16000,
    dias_trabajados: 3,
    descuento: 0,
    neto: 407000.0,
    total_kilos: 380.5
  },
  %{
    recolector: %{codigo: "R03", nombre: "Dora Cardona", alimentacion: true},
    detalle_diario: [
      %{kilos: 140, dia: 1, valor: 140000.0, bonificacion: 8000},
      %{kilos: 85, dia: 2, valor: 85000.0, bonificacion: 0},
      %{kilos: 135, dia: 3, valor: 127500.0, bonificacion: 8000}
    ],
    suma_pesajes: 352500.0,
    total_bonos: 16000,
    dias_trabajados: 3,
    descuento: 36000,
    neto: 332500.0,
    total_kilos: 360
  },
]
```

```

    },
    %{
      recolector: %{
        codigo: "R01",
        nombre: "Luz Marina Ospina",
        alimentacion: true
      },
      detalle_diario: [
        %{kilos: 125, dia: 1, valor: 123000.0, bonificacion: 8000},
        %{kilos: 90, dia: 2, valor: 62999.99999999999, bonificacion: 0}
      ],
      suma_pesajes: 186000.0,
      total_bonos: 8000,
      dias_trabajados: 2,
      descuento: 24000,
      neto: 170000.0,
      total_kilos: 215
    },
    %{
      recolector: %{codigo: "R04", nombre: "Wilson Arango", alimentacion: false},
      detalle_diario: [],
      suma_pesajes: 0,
      total_bonos: 0,
      dias_trabajados: 0,
      descuento: 0,
      neto: 0,
      total_kilos: 0
    }
  ]
}

```

Prueba 2: Reportes.ranking(liquidaciones, campo: :kilos, limite: 3)

```

[
  %{
    recolector: %{
      codigo: "R02",

```

```
    codigo: "R02",
    nombre: "Jhon Fredy Castaño",
    alimentacion: false
  },
  detalle_diario: [
    %{kilos: 145, dia: 1, valor: 150000.0, bonificacion: 8000},
    %{kilos: 125.5, dia: 2, valor: 125500.0, bonificacion: 8000},
    %{kilos: 110, dia: 3, valor: 115500.0, bonificacion: 0}
  ],
  suma_pesajes: 391000.0,
  total_bonos: 16000,
  dias_trabajados: 3,
  descuento: 0,
  neto: 407000.0,
  total_kilos: 380.5
},
%{
  recolector: %{codigo: "R03", nombre: "Dora Cardona", alimentacion: true},
  detalle_diario: [
    %{kilos: 140, dia: 1, valor: 140000.0, bonificacion: 8000},
    %{kilos: 85, dia: 2, valor: 85000.0, bonificacion: 0},
    %{kilos: 135, dia: 3, valor: 127500.0, bonificacion: 8000}
  ],
  suma_pesajes: 352500.0,
  total_bonos: 16000,
  dias_trabajados: 3,
  descuento: 36000,
  neto: 332500.0,
  total_kilos: 360
},
%{
  recolector: %{
    codigo: "R01",
    nombre: "Luz Marina Ospina",
    alimentacion: true
```

```

    alimentacion: true
  },
  detalle_diario: [
    %{kilos: 125, dia: 1, valor: 123000.0, bonificacion: 8000},
    %{kilos: 90, dia: 2, valor: 62999.99999999999, bonificacion: 0}
  ],
  suma_pesajes: 186000.0,
  total_bonos: 8000,
  dias_trabajados: 2,
  descuento: 24000,
  neto: 170000.0,
  total_kilos: 215
}
]

```

Prueba 3: Reportes.ranking(liquidaciones, orden: :asc, campo: :bruto)

```

[
  %{
    recolector: %{codigo: "R04", nombre: "Wilson Arango", alimentacion: false},
    detalle_diario: [],
    suma_pesajes: 0,
    total_bonos: 0,
    dias_trabajados: 0,
    descuento: 0,
    neto: 0,
    total_kilos: 0
  },
  %{
    recolector: %{
      codigo: "R01",
      nombre: "Luz Marina Ospina",
      alimentacion: true
    },
    detalle_diario: [
      %{kilos: 125, dia: 1, valor: 123000.0, bonificacion: 8000},

```

```

    %{kilos: 125, dia: 1, valor: 123000.0, bonificacion: 8000},
    %{kilos: 90, dia: 2, valor: 62999.999999999999, bonificacion: 0}
  ],
  suma_pesajes: 186000.0,
  total_bonos: 8000,
  dias_trabajados: 2,
  descuento: 24000,
  neto: 170000.0,
  total_kilos: 215
},
%{
  recolector: %{codigo: "R03", nombre: "Dora Cardona", alimentacion: true},
  detalle_diario: [
    %{kilos: 140, dia: 1, valor: 140000.0, bonificacion: 8000},
    %{kilos: 85, dia: 2, valor: 85000.0, bonificacion: 0},
    %{kilos: 135, dia: 3, valor: 127500.0, bonificacion: 8000}
  ],
  suma_pesajes: 352500.0,
  total_bonos: 16000,
  dias_trabajados: 3,
  descuento: 36000,
  neto: 332500.0,
  total_kilos: 360
},
%{
  recolector: %{
    codigo: "R02",
    nombre: "Jhon Fredy Castaño",
    alimentacion: false
  },
  detalle_diario: [
    %{kilos: 145, dia: 1, valor: 150000.0, bonificacion: 8000},
    %{kilos: 125.5, dia: 2, valor: 125500.0, bonificacion: 8000},
    %{kilos: 110, dia: 3, valor: 115500.0, bonificacion: 0}
  ],
}
]

```

```

    %{kilos: 145, dia: 1, valor: 150000.0, bonificacion: 8000},
    %{kilos: 125.5, dia: 2, valor: 125500.0, bonificacion: 8000},
    %{kilos: 110, dia: 3, valor: 115500.0, bonificacion: 0}
  ],
  suma_pesajes: 391000.0,
  total_bonos: 16000,
  dias_trabajados: 3,
  descuento: 0,
  neto: 407000.0,
  total_kilos: 380.5
}
]

--- COMPONENTE C.2: Combinar Fincas ---
%{1 => 930.5, 2 => 910.5, 3 => 725, 4 => 0, 5 => 700, 6 => 0, 7 => 300}

```

1. ¿Por qué Elixir usa keyword lists, y no mapas, para pasar opciones a una función?

R/ Elixir utiliza keyword lists porque, a diferencia de los mapas, garantizan que se mantenga el orden de los elementos y permiten tener llaves duplicadas. Además, ofrecen un "azúcar sintáctico" que mejora la legibilidad del código: cuando una keyword list es el último argumento de una función, se pueden omitir los corchetes [] en la llamada.

2. ¿Qué devuelve su función si se llama con campo: :kilos, campo: :neto , y por qué?

R/ Devolverá :kilos. Esto ocurre porque la función utiliza Keyword.get para buscar la llave. En Elixir, si hay llaves repetidas en una keyword list, la función siempre extraerá y devolverá el primer valor que encuentre de izquierda a derecha.

C.2. Combinar reportes de dos fincas

Código que hace que se combinen el reporte de 2 fincas:

```
def combinar_fincas(mapa_finca, finca_vecina) do
  Map.merge(mapa_finca, finca_vecina, fn _dia, kilos_finca,
kilos_vecina ->
    kilos_finca + kilos_vecina
  end)
end
end
```

¿Qué resultado se habría obtenido con Map.merge/2? Habría sobrescrito los kilos de nuestra finca con los de la finca vecina en los días coincidentes. Por ejemplo, en el día 1, en lugar de sumar $410 + 520.5 = 930.5$, el valor resultante habría quedado simplemente en 520.5.

¿Por qué ese resultado es incorrecto? Porque el objetivo es consolidar (sumar) la producción total de ambas fincas, no reemplazar la producción de una con la de la otra. Por eso usamos Map.merge/3, que nos permite declarar una función para sumar los valores cuando la llave choca.

¿Qué pasa con el día 7? Como el día 7 solo existe en el mapa de la finca vecina y no hay conflicto de llaves, Map.merge simplemente inserta el par 7 => 300 directamente en el mapa final combinado.

Parte D. Uso justificado de inteligencia artificial

D.1. Bitácora

Parte	Herramienta	Qué le pidieron	Por qué recurrieron a la IA en ese punto	Cómo verificaron la respuesta	Qué hicieron con ella (la usaron, la corrigieron, la descartaron)
Desarrollo del código	Gemini	Explicación teórica, casos de uso y ejemplos prácticos de las funciones: <code>Enum.filter</code> , <code>Enum.group_by</code> , <code>String.split</code> , <code>Enum.find</code> , <code>Enum.join</code> , <code>Enum.frequencies_by</code> , <code>Map.get</code> , <code>Enum.sort_by</code> , <code>Enum.all?</code> , <code>Enum.any?</code> , <code>Enum.with_index</code> , <code>Enum.max_by</code> , <code>Enum.min_by</code> y <code>Enum.take</code> .	No sabíamos cómo manipular ni transformar eficientemente las colecciones de datos en Elixir para nuestro proyecto. Necesitábamos entender que <code>Enum.filter</code> se usa para filtrar datos que cumplen una condición, <code>Enum.group_by</code> para agrupar elementos por una clave, <code>String.split</code> para separar cadenas de texto, <code>Enum.find</code> para localizar el primer elemento que coincide con un criterio, y <code>Enum.join</code> para unir elementos de una lista en un solo texto. También debíamos aclarar que <code>Enum.frequencies_by</code> sirve para contar las ocurrencias de un dato, <code>Map.get</code> para extraer valores específicos de un diccionario/mapa, <code>Enum.sort_by</code> para ordenar listas basadas en un atributo, y <code>Enum.all?</code> junto con <code>Enum.any?</code> para validar si todos o al menos un elemento cumplen con una regla. Finalmente, necesitábamos saber que <code>Enum.with_index</code> asocia un número de posición a cada elemento, <code>Enum.max_by</code> y <code>Enum.min_by</code> extraen el valor más alto o más bajo según un parámetro, y <code>Enum.take</code> permite obtener una cantidad exacta de elementos de una lista.	Ejecutando los fragmentos de código proporcionados directamente en la consola interactiva de Elixir para observar el comportamiento con datos de prueba, y validando los retornos dentro de las funciones del proyecto.	La estudiamos, vimos para qué servía cada diferente <code>enum</code> y la implementamos en nuestro parcial 1 el cual nos sirvió de mucha ayuda en la solución del código.
Generación de Reportes	Gemini / Documentación	Cómo obtener solo los primeros N elementos de una lista ya ordenada para el sistema de ranking (<code>Enum.take/2</code>).	No sabíamos cómo limitar la cantidad de resultados de una lista para hacer un "Top 3". Necesitábamos entender que <code>Enum.take/2</code> extrae exactamente la cantidad de elementos solicitados desde el inicio de una colección.	Ejecutando la función en IEx y revisando la salida de <code>Reportes.ranking/2</code> en la terminal.	La estudiamos y la implementamos en la función <code>ranking/2</code> de nuestro Parcial 1 para extraer los mejores recolectores según el límite especificado en las opciones.
Generación de Reportes	Gemini / Documentación	Cómo combinar dos mapas sumando sus valores si las claves coinciden (<code>Map.merge/3</code> con función de resolución).	Necesitábamos entender cómo unir dos mapas distintos (por ejemplo, <code>mapa_finca</code> y <code>finca_vecina</code>) de forma que, si ambos mapas tenían la misma clave, sus valores se sumaran en lugar de que uno sobrescribiera al otro.	Probando con mapas pequeños en IEx y verificando que la suma de kilos funcionara al combinar fincas.	La estudiamos, vimos para qué servía y la implementamos en la función <code>combinar_fincas/2</code> del parcial.
Conversión de Tipos	Gemini / Documentación	Cómo validar si un dato es un número (entero o decimal) sin importar su formato (<code>is_number/1</code>).	Necesitábamos entender cómo comprobar eficientemente en una sola validación (guardia) si una variable de peso (kilos) o porcentaje (verdes) era un valor numérico válido, ya fuera entero o flotante.	Usando la función dentro del bloque de validación (<code>validar_kilos/1</code>) y enviando textos y números para ver su comportamiento.	La estudiamos y la usamos en el módulo <code>Validacion</code> para garantizar la integridad de los datos de entrada antes de agregarlos al sistema.
Validación de Datos	Gemini / Documentación	Cómo validar específicamente si una variable es un número entero (<code>is_integer/1</code>).	Necesitábamos entender cómo asegurarnos de que la variable <code>dia</code> fuera estrictamente un número entero y no un decimal, ya que en la lógica del negocio los días (del 1 al 6) no pueden tener fracciones.	Verificando su funcionamiento en el módulo <code>Validacion</code> , pasando valores como 1.5 que debían ser rechazados.	La estudiamos y la implementamos en la función <code>validar_dia/1</code> de nuestro Parcial 1 para restringir correctamente los días de la semana.
Cálculo de Variables	Gemini / Documentación	Cómo obtener rápidamente el tamaño o la cantidad de claves de un mapa (<code>map_size/1</code>).	Necesitábamos entender cómo contar cuántos días diferentes había trabajado un recolector después de haber agrupado sus pesajes por día (ya que esto nos devolvía un mapa, no una lista).	Ejecutando la función sobre el mapa <code>pesajes_por_dia</code> para asegurar que el conteo de días trabajados fuera preciso.	La estudiamos y la usamos en la función <code>liquidar_recolector/2</code> del parcial para calcular correctamente los días trabajados y aplicar el descuento de alimentación.
Extracción de Tuplas	Gemini / Documentación	Cómo acceder a un elemento específico dentro de una tupla por su posición (<code>elem/2</code>).	No sabíamos cómo extraer eficientemente el segundo valor (el de los kilos) de las tuplas que generábamos temporalmente al buscar el mayor valor o iterar sobre agrupaciones.	Ejecutando pruebas en la consola interactiva con tuplas dummy y verificando el retorno de la función.	La estudiamos y la implementamos en la función <code>r5_mejores_dias/2</code> para extraer el valor máximo (<code>max_kilos</code>) de la tupla retornada por <code>Enum.max_by</code> .

D.2. Un error de la IA

Durante el desarrollo del punto B.5 (Interacción con el usuario), le pedimos a la IA sugerencias sobre cómo convertir los datos ingresados por la consola (que entran como texto String) a números decimales para poder procesar los kilos y los porcentajes de verdes.

Respuesta original de la IA:

La IA nos sugirió usar `String.to_float/1` envuelto en un bloque `try/rescue` para evitar que el programa fallara y se detuviera si el usuario escribía letras en lugar de números:

```
def parsear_numero(string) do
  try do
    {:ok, String.to_float(string)}
  rescue
    ArgumentError -> {:error, :invalido}
  end
end
```

Cómo notamos el problema: Al revisar las instrucciones y restricciones generales del parcial, notamos que el uso de la estructura `try / rescue` estaba estrictamente prohibido y sería calificado con cero. En Elixir, el manejo de errores esperado para este nivel debe hacerse mediante tuplas (`{:ok, valor}` o `{:error, motivo}`) y pattern matching. Además, nos dimos cuenta de que la IA estaba usando una función fuera del alcance recomendado, ya que debíamos utilizar `float.parse`.

Código final:

Descartamos la sugerencia de la IA, evitamos el uso del `try/rescue` y construimos nuestra propia función de conversión utilizando `case`, la cual maneja correctamente las tuplas que devuelve `Float.parse`:

```
defp parse_numero(string) do
  case Float.parse(string) do
    {num, ""} -> {:ok, num}
    _ ->
      case Integer.parse(string) do
```

```
{num, ""} -> {:ok, num * 1.0}
_ -> {:error, :invalido}

end

end

end
```

D.3. Reflexión del grupo

1. ¿En qué partes la IA les ayudó a entender algo que no sabían, y en cuáles se limitó a darles código?

R/ La ia fue una herramienta clave para interiorizar el paradigma de la programación funcional, especialmente el manejo de colecciones inmutables. Al nosotros venir de lenguajes de programación imperativos, nuestro primer instinto para calcular totales era intentar crear variables globales e iterar con ciclos clásicos para ir acumulando sumas. La IA nos ayudó a comprender profundamente la mecánica funcional detrás de las funciones de orden superior, explicándonos de forma didáctica cómo usar Enum.reduce para acumular valores y Enum.group_by para estructurar datos intermedios en mapas. Por otro lado, la IA se limitó a darnos código puro en tareas de formato y sintaxis muy específica, como por ejemplo la manera exacta de utilizar :erlang.float_to_binary(valor, decimals: 2) para imprimir el dinero con dos decimales evitando la notación científica, o la estructura sintáctica del bloque with para encadenar las validaciones.

2. ¿Qué partes del proyecto decidieron hacer sin IA y por qué?

R/ Decidimos implementar la lógica central de negocio puramente "a mano". Esto abarca los módulos de validacion (las reglas de rechazo) y liquidacion (los cálculos matemáticos de bonificaciones, castigos por porcentajes de verdes y descuentos por alimentación). Tomamos esta decisión porque estas reglas son altamente específicas del contexto de este parcial. Temíamos que, si le dábamos el texto del problema a la IA, esta pudiera sobrecomplicar la lógica, interpretar mal los porcentajes o utilizar funciones avanzadas fuera del alcance de la materia. Preferimos traducir nosotros mismos cada viñeta del documento a funciones sencillas usando condicionales clásicos y operadores matemáticos básicos, garantizando así un control absoluto sobre la exactitud de los salarios.

3.Si tuvieran que hacer el proyecto de nuevo sin ninguna herramienta de IA, ¿qué parte les costaría más?

Sin duda, la parte que más nos costaría sería la elaboración de los reportes estadísticos complejos, específicamente el Reporte 5 (Mejor recolector de cada día). Lograr la lógica de tomar una lista plana de pesajes, agruparlos primero por día, luego reagruparlos internamente por recolector, sumar sus kilos parciales y finalmente usar `Enum.max_by` para hallar al ganador (considerando además el manejo de posibles empates), habría sido un reto inmenso. Sin la IA para explicarnos cómo encadenar estas transformaciones paso a paso con el operador pipe (`|>`), habríamos tenido que recurrir a muchísimo ensayo y error, leyendo extensa documentación, lo cual nos habría tomado días de depuración para lograr que las estructuras de datos coincidieran.